



# AI SECURITY AUDIT REPORT

|               |                                                                           |
|---------------|---------------------------------------------------------------------------|
| Target System | ProjecteAsix                                                              |
| Target IP(s)  | 127.0.0.1, 172.17.0.1, 192.168.1.72, fe80::be24:11ff:fe54:638a, localhost |
| Platform      | Linux 6.8.0-111-generic                                                   |
| Audit Date    | 2026-05-19                                                                |
| Generated     | Cybershield AI                                                            |
| Report ID     | AI-1779206661                                                             |

**CONFIDENTIAL**

Cybershield AI Solutions  
AI-Powered Security Analysis

# Executive Summary

This AI-generated security analysis was produced by the Cybershield AI platform based on comprehensive scans.



## Risk Assessment

System risk level: HIGH

As your cybersecurity expert assistant, I've analyzed the provided security audit data for "ProjecteAsix". Here's an executive summary designed for clarity and actionable insights:

Executive Summary for ProjecteAsix

This security audit reveals a HIGH risk level for the "ProjecteAsix" system, primarily driven by a significant number of high-severity vulnerabilities and a lack of recent package updates. Immediate attention is required to mitigate potential security breaches and ensure the integrity of the system.

### Top 5 Findings

- **Extensive High-Severity Vulnerabilities:** The system exhibits a concerning 374 high-severity vulnerabilities. This indicates a broad attack surface and a high likelihood of exploitation by malicious actors.
- **Outdated Software Packages:** 15 packages require updates. Running outdated software is a common entry point for attackers, as known vulnerabilities in older versions are often exploited.
- **Elevated SSH Brute-Force Attempts:** 15 SSH login failures were recorded. This suggests the system is being targeted by brute-force attacks, attempting to gain unauthorized access via SSH.
- **Presence of Suspicious Processes:** 4 suspicious processes were detected. These could represent malware, unauthorized applications, or compromised services running on the system, potentially leading to data exfiltration or system disruption.
- **Overall High Risk Level:** The combination of these factors has resulted in an overall HIGH risk level for "ProjecteAsix", demanding urgent remediation efforts.

### Top 5 Recommendations

- **Prioritize Vulnerability Remediation:** Immediately address the 374 high-severity vulnerabilities. This should involve a phased approach, starting with the most critical and easily exploitable vulnerabilities.
- **Action:** Consult the detailed vulnerability report to identify specific CVEs and their associated remediation steps. For many vulnerabilities, this will involve applying security patches or updating software.
- **Update All Outdated Packages:** Promptly update the 15 packages that require updates. This is a fundamental security practice to close known security gaps.
- **Action:** For Debian/Ubuntu-based systems, use: `sudo apt update && sudo apt upgrade -y`. For RHEL/CentOS/Fedora, use: `sudo yum update -y` or `sudo dnf update -y`.

- **Strengthen SSH Security:** Implement robust measures to protect against SSH brute-force attacks.
- **Action:**
- **Disable root login:** Edit `/etc/ssh/sshd_config` and set `PermitRootLogin no`.
- **Use strong passwords and consider key-based authentication:** Enforce complex password policies and migrate to SSH keys for authentication.
- **Implement rate limiting:** Use tools like `fail2ban` to automatically block IP addresses that exhibit suspicious login activity. Install with `sudo apt install fail2ban` or `sudo yum install fail2ban` and configure its rules.
- **Investigate and Neutralize Suspicious Processes:** Thoroughly investigate the **4 suspicious processes** to determine their origin and purpose.
- **Action:** Use commands like `ps aux` or `top` to identify running processes. For each suspicious process, research its name and associated user. If confirmed as malicious, terminate the process immediately using `sudo kill -9` and then remove any associated files or malware. Consider running a full system malware scan.
- **Develop a Continuous Security Monitoring Strategy:** Establish ongoing monitoring to detect and respond to security threats proactively.
- **Action:** Implement regular vulnerability scanning, log analysis, and intrusion detection systems. Schedule periodic security audits to ensure continuous improvement and maintain a lower risk posture.

## Trivy - Vulnerabilities (HIGH/CRITICAL)

### AI Analysis:

#### Security Audit Analysis

This audit reveals a **critical risk level** due to multiple high-severity vulnerabilities affecting the Linux kernel. The most critical CVEs are **CVE-2026-23112** (nvmet-tcp bounds check) and **CVE-2026-23231** (use-after-free in nf\_tables\_addchain), both of which can lead to privilege escalation or denial of service. **Immediate patching** of these kernel components is paramount to mitigate these significant security threats.

#### Remediation Steps

To address these vulnerabilities, it is crucial to update your Linux kernel to a patched version. The specific commands will depend on your Linux distribution, but generally, you would use your distribution's package manager.

For Debian/Ubuntu-based systems, you would typically run:

```
sudo apt update
sudo apt upgrade linux-image-generic linux-headers-generic
```

For Red Hat/CentOS/Fedora-based systems, you would use:

```
sudo yum update kernel
or

sudo dnf update kernel
```

After applying the updates, a system reboot will be necessary for the new kernel to take effect. It is also recommended to regularly scan your systems for vulnerabilities and keep all software packages up-to-date to maintain a strong security posture.

| Severity | Package                         | Installed     | Fixed Version | CVE ID         | Title                                                                               |
|----------|---------------------------------|---------------|---------------|----------------|-------------------------------------------------------------------------------------|
| HIGH     | linux-headers-6.8.0-110         | 6.8.0-110.110 | 6.8.0-111.111 | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH     | linux-headers-6.8.0-110         | 6.8.0-110.110 | 6.8.0-111.111 | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service via use-after-free in ... |
| HIGH     | linux-headers-6.8.0-110-generic | 6.8.0-110.110 | 6.8.0-111.111 | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH     | linux-headers-6.8.0-110-generic | 6.8.0-110.110 | 6.8.0-111.111 | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service                           |

via use-after-free  
in ...

|      |                                       |               |                                                         |                |                                                                                     |
|------|---------------------------------------|---------------|---------------------------------------------------------|----------------|-------------------------------------------------------------------------------------|
| HIGH | linux-modules-6.8.0-110-generic       | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH | linux-modules-6.8.0-110-generic       | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service via use-after-free in ... |
| HIGH | linux-modules-extra-6.8.0-110-generic | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH | linux-modules-extra-6.8.0-110-generic | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service via use-after-free in ... |
| HIGH | linux-tools-6.8.0-110                 | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH | linux-tools-6.8.0-110                 | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service via use-after-free in ... |
| HIGH | linux-tools-6.8.0-110-generic         | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23112 | kernel: nvmet-tcp: add bounds checks in nvmet_tcp_build_pdu_iovec...                |
| HIGH | linux-tools-6.8.0-110-generic         | 6.8.0-110.110 | 6.8.0-111.111                                           | CVE-2026-23231 | kernel: kernel: Privilege escalation or denial of service via use-after-free in ... |
| HIGH | cross-spawn                           | 7.0.3         | 7.0.5, 6.0.6                                            | CVE-2024-21538 | cross-spawn: regular expression denial of service...                                |
| HIGH | glob                                  | 10.4.2        | 11.1.0, 10.5.0                                          | CVE-2025-64756 | glob: glob: Command Injection Vulnerability via Malicious Filenames...              |
| HIGH | minimatch                             | 9.0.5         | 10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3 | CVE-2026-26996 | minimatch: minimatch: Denial of Service via specially crafted glob patterns...      |

|      |              |        |                                                         |                     |                                                                                     |
|------|--------------|--------|---------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------|
| HIGH | minimatch    | 9.0.5  | 10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3 | CVE-2026-27903      | minimatch: minimatch: Denial of Service due to unbounded recursive backtracking ... |
| HIGH | minimatch    | 9.0.5  | 10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4 | CVE-2026-27904      | minimatch: Minimatch: Denial of Service via catastrophic backtracking in glob ex... |
| HIGH | tar          | 6.2.1  | 7.5.3                                                   | CVE-2026-23745      | node-tar: tar: node-tar: Arbitrary file overwrite and symlink poisoning via unsa... |
| HIGH | tar          | 6.2.1  | 7.5.4                                                   | CVE-2026-23950      | node-tar: tar: node-tar: Arbitrary file overwrite via Unicode path collision rac... |
| HIGH | tar          | 6.2.1  | 7.5.7                                                   | CVE-2026-24842      | node-tar: tar: node-tar: Arbitrary file creation via path traversal bypass in ha... |
| HIGH | tar          | 6.2.1  | 7.5.8                                                   | CVE-2026-26960      | node-tar: node-tar: Arbitrary file read/write via malicious archive hardlink cre... |
| HIGH | tar          | 6.2.1  | 7.5.10                                                  | CVE-2026-29786      | node-tar: hardlink path traversal via drive-relative linkpath...                    |
| HIGH | tar          | 6.2.1  | 7.5.11                                                  | CVE-2026-31802      | tar: tar: File overwrite via drive-relative symlink traversal...                    |
| HIGH | Scrapy       | 2.14.1 | None                                                    | CVE-2017-14158      | Scrapy 1.4 allows remote attackers to cause a denial of service (memor .....        |
| HIGH | Scrapy       | 2.14.1 | 2.14.2                                                  | GHSA-cwxj-rr6w-m6w7 | Scrapy: Arbitrary Module Import via Referrer-Policy Header in RefererMiddleware..   |
| HIGH | cryptography | 46.0.3 | 46.0.5                                                  | CVE-2026-26007      | cryptography: cryptography Subgroup Attack Due to Missing                           |

|      |              |        |        |                |                                                                                     |
|------|--------------|--------|--------|----------------|-------------------------------------------------------------------------------------|
|      |              |        |        |                | Subgroup Validation fo...                                                           |
| HIGH | cryptography | 46.0.3 | 46.0.5 | CVE-2026-26007 | cryptography: cryptography Subgroup Attack Due to Missing Subgroup Validation fo... |
| HIGH | lxml         | 6.0.2  | 6.1.0  | CVE-2026-41066 | lxml: python: lxml: Information disclosure via untrusted XML input leading to lo... |
| HIGH | lxml         | 6.0.2  | 6.1.0  | CVE-2026-41066 | lxml: python: lxml: Information disclosure via untrusted XML input leading to lo... |
| HIGH | pillow       | 11.3.0 | 12.1.1 | CVE-2026-25990 | pillow: Pillow: Out-of-bounds Write via Specially Crafted PSD Image...              |

Showing top 30 of 200 total vulnerabilities.



## Network Security Assessment (127.0.0.1)



### Executive Summary

**Hosts Scanned:** 1 up, 0 down

**Total Open Ports:** 5

 **Critical Vulnerabilities:** 0 CVEs detected

 **Critical (CVSS 9.0+):** 0

**High Severity (CVSS 7.0-8.9):** 0



### Detected Vulnerabilities (Top 50)

| Host                        | Port | Service | CVSS | CVE ID | Exploit |
|-----------------------------|------|---------|------|--------|---------|
| No vulnerabilities detected |      |         |      |        |         |



### Open Ports

| Host      | Port/Protocol | State | Service      | Security Level |
|-----------|---------------|-------|--------------|----------------|
| 127.0.0.1 | 22/tcp        | open  | ssh          | Safe           |
| 127.0.0.1 | 80/tcp        | open  | http         | Safe           |
| 127.0.0.1 | 139/tcp       | open  | netbios-ssn  | Dangerous      |
| 127.0.0.1 | 443/tcp       | open  | https        | Safe           |
| 127.0.0.1 | 445/tcp       | open  | microsoft-ds | Dangerous      |



#### AI Security Analysis:

Gemini API Error: 503 - This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.

## Upgradable Packages (15 total)

AI Analysis:

No critical security packages identified.

| Package                    | Current | Candidate |
|----------------------------|---------|-----------|
| No packages data available |         |           |

Showing top 50 of 15 upgradable packages. Security-related packages flagged for priority.

## SSH Failed Login Attempts (15 total)

### AI Analysis:

As a cybersecurity expert assistant, I can help you analyze this SSH login failure report.

#### Security Assessment

The report indicates **15 failed SSH login attempts originating from 0 unique IP addresses**. This is a peculiar situation. Typically, failed login attempts are associated with specific IP addresses that are trying to gain unauthorized access. The fact that there are failed attempts but no identified IP addresses suggests a few possibilities:

- **Internal Source:** The attempts might be originating from within your internal network. This could be due to misconfigured internal systems, automated scripts, or even a compromised internal machine.
- **IP Address Masking/Obfuscation:** The source IPs might be intentionally hidden or obfuscated, making them difficult to track. This is less common for brute-force attacks but could be a sign of a more sophisticated attacker.
- **Logging Anomaly:** There might be an issue with how your logging system is capturing and reporting IP addresses for SSH failures. The system might be failing to log the IP correctly in these instances.
- **Very Short-Lived Connections:** The attempts might be so brief that the IP address isn't being properly recorded before the connection is terminated.

While the number of failed attempts (15) is relatively low, the lack of identifiable source IPs is a **potential red flag**. It prevents you from immediately blocking malicious actors and requires further investigation to understand the root cause.

#### Recommendations

Here are some recommendations to address this situation and improve your SSH security:

##### 1. Investigate the Source of the Failed Attempts

- **Examine SSH Logs in Detail:** The provided report is a summary. You need to dive deeper into your SSH logs to get more context.
- **Command (Linux/macOS):**

```
sudo grep "Failed password" /var/log/auth.log 0r /var/log/secure on some systems
```

Look for patterns, timestamps, and any other information that might be present even if the IP is missing.

- **Check for Internal IP Ranges:** If you suspect an internal source, review logs for any internal IP addresses that might be present.
- **Review Network Traffic:** If possible, analyze network traffic logs around the time of these failed attempts to see if you can identify any unusual connections to your SSH port.

##### 1. Enhance SSH Security Configurations

- **Disable Root Login:** Never allow direct SSH login as the root user.
- **Remediation:** Edit your SSH server configuration file (/etc/ssh/sshd\_config) and ensure the following line is present and uncommented:

```
PermitRootLogin no
```

Then, restart the SSH service:

```
sudo systemctl restart sshd Or sudo service ssh restart
```

- **Use SSH Key-Based Authentication:** This is significantly more secure than password-based authentication.

- **Remediation:**

1. **Generate SSH Keys:** On your client machine: `ssh-keygen -t rsa -b 4096`
2. **Copy Public Key to Server:** `ssh-copy-id user@your_server_ip`
3. **Disable Password Authentication:** Edit `/etc/ssh/sshd_config` on the server and set:

```
PasswordAuthentication no
```

1. Restart SSH service.

- **Implement Fail2Ban:** This is a crucial intrusion prevention framework that scans log files and bans IPs that show malicious signs (like too many password failures).

- **Remediation (Debian/Ubuntu):**

```
sudo apt update
```

```
sudo apt install fail2ban
```

- **Remediation (CentOS/RHEL):**

```
sudo yum install epel-release
```

```
sudo yum install fail2ban
```

- **Configuration:** After installation, you'll want to configure `jail.local` (copy `jail.conf` to `jail.local` to avoid overwriting during updates) to enable the SSH jail and set parameters like `bantime` and `maxretry`.

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

```
sudo nano /etc/fail2ban/jail.local
```

Ensure the `[sshd]` section is enabled and configured appropriately.

```
[sshd]
```

```
enabled = true
```

```
port = ssh
```

```
filter = sshd
```

```
logpath = %(sshd_log)s
```

```
maxretry = 3 Adjust as needed
```

```
bantime = 1h Adjust as needed
```

Then start and enable Fail2Ban:

```
sudo systemctl start fail2ban
```

```
sudo systemctl enable fail2ban
```

- **Change Default SSH Port:** While not a foolproof security measure, changing the default port (22) can reduce automated scans.
- **Remediation:** Edit `/etc/ssh/sshd_config` and change the Port directive:

```
Port 2222 Or any other non-standard port
```

Remember to update your firewall rules to allow traffic on the new port and restart the SSH service.

- **Limit SSH Access by IP Address:** If you have a static set of IPs that should access your server, restrict SSH access to only those IPs.
- **Remediation:** Edit `/etc/ssh/sshd_config` and use `AllowUsers` or `AllowGroups` with IP restrictions, or configure your firewall (e.g., `ufw`, `firewalld`, `iptables`) to only allow SSH connections from specific IPs.
- **Example using ufw:**

```
sudo ufw allow from to any port 22
```

```
sudo ufw enable
```

## 1. Monitor and Alert

- **Set Up Regular Log Review:** Make log analysis a routine part of your security operations.
- **Configure Alerts:** If your security monitoring tools can detect patterns of failed SSH logins (even without specific IPs), set up alerts to notify you immediately.

The absence of unique IPs for these failed attempts is the most concerning aspect. Prioritize investigating the source to ensure it's not an internal issue or a sign of a more advanced attack. Implementing robust SSH security measures like key-based authentication and Fail2Ban will significantly harden your server against brute-force attacks.

| Timestamp | IP Address |
|-----------|------------|
|-----------|------------|

Running Services (149 total)

AI Analysis:

Gemini API Error: 503 - This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.

| Service | State | Description |
|---------|-------|-------------|
|---------|-------|-------------|

## Suspicious Processes (4 detected)

### AI Analysis:

As a cybersecurity expert assistant, I can help you analyze these suspicious processes. It's good that you're being vigilant and looking into unusual activity on your system. Let's break down each process and provide a security assessment.

## Security Assessment of Suspicious Processes

Here's an analysis of the processes you've provided:

- **polkitd (PID: 720, Executable: /usr/lib/polkit-1/polkitd)**
- **Description:** polkitd is the PolicyKit daemon. It's a crucial system component responsible for managing privileges and allowing unprivileged applications to request elevated privileges for specific tasks. It acts as an intermediary between applications and the system's authorization mechanisms.
- **Security Assessment:**
- **Legitimate Process:** polkitd is a **legitimate and essential system process** on most Linux distributions. It's not inherently suspicious.
- **Potential for Abuse:** However, like any powerful system component, it **can be exploited** if vulnerabilities exist within PolicyKit itself or if it's misconfigured. Malware or attackers might try to leverage polkitd to gain elevated privileges on your system.
- **Suspicion Trigger:** If you're seeing this process in an unexpected context, or if it's consuming excessive resources, it might warrant further investigation.
- **Remediation/Further Investigation:**
- **Verify System Integrity:** Ensure your system is up-to-date. Run:

```
sudo apt update && sudo apt upgrade
```

 For Debian/Ubuntu

```
sudo yum update
```

For RHEL/CentOS/Fedora

- **Check PolicyKit Configuration:** While complex, you can review PolicyKit rules in `/etc/polkit-1/rules.d/` and `/usr/share/polkit-1/actions/`. Look for any unusual or overly permissive rules.
- **Monitor Resource Usage:** Use tools like `top` or `htop` to monitor `polkitd`'s CPU and memory usage. Unusually high usage could indicate a problem.
- **unattended-upgr (PID: 738, Executable: /usr/bin/python3.12)**
- **Description:** `unattended-upgr` is the **unattended-upgrades** package, which automatically installs security updates and other critical updates on your system. It's typically run by `cron` or `systemd` timers.
- **Security Assessment:**
- **Legitimate Process:** This is a **highly legitimate and recommended process** for maintaining system security. It ensures your system is patched against known vulnerabilities.
- **Suspicion Trigger:** The only reason this might appear "suspicious" is if you weren't expecting it to run, or if you have disabled automatic updates and it's still active. It's also important to ensure the executable path is correct and not pointing to a malicious script disguised as `unattended-upgrades`.

- **Remediation/Further Investigation:**
- **Confirm Configuration:** Check the configuration of unattended-upgrades. The main configuration file is usually `/etc/apt/apt.conf.d/50unattended-upgrades`. Ensure it's configured as you expect.
- **Review Logs:** Examine the logs for unattended-upgrades to see what actions it has taken. These are typically found in `/var/log/unattended-upgrades/`.
- **Verify Python Version:** While `python3.12` is a specific version, it's generally fine. The key is that it's the legitimate unattended-upgrades script being executed by the correct Python interpreter.
- **dockerd (PID: 862, Executable: /usr/bin/dockerd)**
- **Description:** `dockerd` is the **Docker daemon**. It's the background service that manages Docker containers, images, networks, and volumes.
- **Security Assessment:**
- **Legitimate Process:** If you are using Docker, `dockerd` is a **legitimate and essential process**.
- **Potential for Abuse:** Docker itself, if not configured securely, can introduce security risks. This includes:
- **Exposed Docker Socket:** If the Docker daemon's socket (`/var/run/docker.sock`) is accessible to unprivileged users, they can gain root privileges.
- **Insecure Container Images:** Running containers from untrusted or vulnerable images.
- **Misconfigured Networks:** Exposing container services unnecessarily.
- **Suspicion Trigger:** If you do not use Docker on this system, then the presence of `dockerd` is **highly suspicious** and indicates unauthorized software installation.
- **Remediation/Further Investigation:**
- **If You Use Docker:**
- **Secure the Docker Daemon:** Ensure `/var/run/docker.sock` is only accessible by the `docker` group or `root`. Avoid giving broad access.
- **Regularly Scan Images:** Use tools like Trivy or Clair to scan your Docker images for vulnerabilities.
- **Review Container Security:** Implement best practices for container security, such as running containers as non-root users and limiting their capabilities.
- **If You Do NOT Use Docker:**
- **Uninstall Docker:** This is the primary remediation.

```
sudo apt remove docker docker-engine docker.io containerd runc For Debian/Ubuntu
```

```
sudo systemctl stop dockerd
```

```
sudo systemctl disable dockerd
```

- **Investigate Installation:** Determine how Docker was installed on your system. This could be a sign of a compromise.
- **python3 (PID: 29220, Executable: /usr/bin/python3.12)**
- **Description:** This is a generic Python 3 interpreter. Many legitimate system processes and user-installed applications use Python.
- **Security Assessment:**
- **Legitimate Process:** Python is a widely used language, so seeing a `python3` process is **very common**.

- **Potential for Abuse:** The suspicion arises from **what the Python script is doing**. A malicious script could be disguised as a legitimate Python application.
- **Suspicion Trigger:** If this process is running unexpectedly, consuming excessive resources, or if you can't identify its purpose, it warrants further investigation. The fact that it's running alongside `unattended-upgr` might suggest it's related to system updates, but it's not guaranteed.
- **Remediation/Further Investigation:**
- **Identify the Parent Process:**

| PID   | Name            | User    | Executable                |
|-------|-----------------|---------|---------------------------|
| 720   | polkitd         | polkitd | /usr/lib/polkit-1/polkitd |
| 738   | unattended-upgr | root    | /usr/bin/python3.12       |
| 862   | dockerd         | root    | /usr/bin/dockerd          |
| 29220 | python3         | root    | /usr/bin/python3.12       |

## SSH-Audit (127.0.0.1)

Here's a security assessment of your SSH audit output for 127.0.0.1:

SSH Audit Assessment for 127.0.0.1

This audit provides a good overview of your SSH server's configuration. Overall, it appears to be reasonably secure, but there are a few areas that warrant attention.

### General Configuration

- **Banner:** SSH-2.0-OpenSSH\_9.6p1 Ubuntu-3ubuntu13.16
- **Assessment:** This is the standard SSH banner. It's good practice to have a banner, but it should *\*not\** reveal specific version information to avoid aiding attackers in identifying known vulnerabilities.
- **Software:** OpenSSH 9.6p1
- **Assessment:** You are running a recent version of OpenSSH. This is positive as it likely includes security patches.
- **Compatibility:** OpenSSH 8.5+, Dropbear SSH 2018.76+
- **Assessment:** This indicates good compatibility with modern SSH clients.
- **Compression:** enabled (zlib@openssh.com)
- **Assessment:** Compression can be a double-edged sword. While it can improve performance, it can also be a vector for certain types of attacks (e.g., CRIME/BREACH if used with TLS, though less of a direct concern for SSH itself). For most modern SSH deployments, disabling compression is often recommended for a slightly increased security posture, as the performance gains are often marginal.

### Key Exchange Algorithms

This section is critical for establishing secure SSH connections. The audit highlights both recommended and problematic algorithms.

#### • Recommended Algorithms:

- `sntrup761x25519-sha512@openssh.com` : Available since OpenSSH 8.5. This is a modern and secure algorithm.
- `curve25519-sha256` : Available since OpenSSH 7.4 and Dropbear SSH 2018.76. This is a widely used and secure algorithm. It's also noted as a default key exchange since OpenSSH 6.4.
- `curve25519-sha256@libssh.org` : Available since OpenSSH 6.4 and Dropbear SSH 2013.62. Similar to the above, this is a good option.
- `diffie-hellman-group-exchange-sha256 (3072-bit)` : Available since OpenSSH 4.4. This is a strong Diffie-Hellman group. The note about the fallback mechanism is important: modern clients will use 3072-bit, while very old clients might use 2048-bit. Disabling the fallback would require recompiling, which is generally not feasible or necessary for most users.

#### • Problematic Algorithms (Marked as [fail]):

- `ecdh-sha2-nistp256`
- `ecdh-sha2-nistp384`
- `ecdh-sha2-nistp521`
- **Assessment:** These algorithms are flagged because they use elliptic curves that are suspected of being backdoored by the U.S. National Security Agency (NSA). While the evidence for a widespread, practical backdoor is debated, it's a prudent security measure to disable them if stronger, non-suspect alternatives are available. The audit correctly identifies these as failing.

## Recommendations and Remediation

### 1. Disable Suspect ECDH Algorithms:

- **Reasoning:** To mitigate potential risks associated with suspected NSA backdoors in the NIST P-curves.
- **Action:** You need to configure your SSH server to explicitly disallow these algorithms. This is done by modifying the `sshd_config` file.
- **Command/Configuration:**
- Edit the SSH server configuration file:

```
sudo nano /etc/ssh/sshd_config
```

- Add or modify the `KexAlgorithms` line to exclude the problematic ones. A good, modern configuration would look something like this (ensure you include the ones you want to keep):

```
KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org,diffie-hellman-group-exchange-sha256
```

\*Note: The order matters. Algorithms listed earlier are preferred.\*

- After saving the file, restart the SSH service for the changes to take effect:

```
sudo systemctl restart sshd
```

- **Verification:** Re-run the audit tool after restarting SSH to confirm these algorithms are no longer listed as available or are marked as disabled.

### 1. Consider Disabling Compression:

- **Reasoning:** While not a critical vulnerability in SSH itself, disabling compression can slightly reduce the attack surface and is often recommended for maximum security.
- **Action:** Modify the `sshd_config` file.
- **Command/Configuration:**
- In your `sshd_config` file, add or modify the `Compression` directive:

```
Compression no
```

- Restart the SSH service:

```
sudo systemctl restart sshd
```

### 1. Review SSH Banner:

- **Reasoning:** To avoid revealing specific SSH version information, which can be used by attackers to target known vulnerabilities.
- **Action:** Modify the `sshd_config` file to use a generic banner.
- **Command/Configuration:**
- In your `sshd_config` file, you can set a custom banner message. First, create a banner file (e.g., `/etc/ssh/banner.txt`):

```
echo "Welcome to the secure server. Unauthorized access is prohibited." | sudo tee /etc/ssh/banner.txt
```

- Then, in `sshd_config`, add or modify the `Banner` directive:

```
Banner /etc/ssh/banner.txt
```

- Restart the SSH service:

```
sudo systemctl restart sshd
```

- \*Alternatively, if you don't want a custom banner, you can disable it entirely by commenting out or removing the `Banner` line.\*

## 1. Keep OpenSSH Updated:

- **Reasoning:** You are currently on a good version (9.6

```
# general
(gen) banner: SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.16
(gen) software: OpenSSH 9.6p1
(gen) compatibility: OpenSSH 8.5+, Dropbear SSH 2018.76+
(gen) compression: enabled (zlib@openssh.com)

# key exchange algorithms
(kex) sntrup761x25519-sha512@openssh.com -- [info] available since OpenSSH 8.5
(kex) curve25519-sha256 -- [info] available since OpenSSH 7.4, Dropbear SSH 2018.76
-- [info] default key exchange since OpenSSH 6.4
(kex) curve25519-sha256@libssh.org -- [info] available since OpenSSH 6.4, Dropbear SSH 2013.62
-- [info] default key exchange since OpenSSH 6.4
(kex) ecdh-sha2-nistp256 -- [fail] using elliptic curves that are suspected as being backdoor
-- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex) ecdh-sha2-nistp384 -- [fail] using elliptic curves that are suspected as being backdoor
-- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex) ecdh-sha2-nistp521 -- [fail] using elliptic curves that are suspected as being backdoor
-- [info] available since OpenSSH 5.7, Dropbear SSH 2013.62
(kex) diffie-hellman-group-exchange-sha256 (3072-bit) -- [info] available since OpenSSH 4.4
-- [info] OpenSSH's GEX fallback mechanism was triggered d
(kex) diffie-hellman-group16-sha512 -- [info] available since OpenSSH 7.3, Dropbear SSH 2016.73
(kex) diffie-hellman-group18-sha512 -- [info] available since OpenSSH 7.3
(kex) diffie-hellman-group14-sha256 -- [warn] 2048-bit modulus only provides 112-bits of symmetric stren
-- [info] available since OpenSSH 7.3, Dropbear SSH 2016.73
(kex) ext-info-s -- [info] pseudo-algorithm that denotes the peer supports RFC8308 ex
(kex) kex-strict-s-v00@openssh.com -- [info] pseudo-algorithm that denotes the peer supports a stricter

# host-key algorithms
(key) rsa-sha2-512 (3072-bit) -- [info] available since OpenSSH 7.2
(key) rsa-sha2-256 (3072-bit) -- [info] available since OpenSSH 7.2
```

**Disclaimer:** AI-generated report. Review findings before taking action. False positives may occur.